Adaptar el Portal Deportivo a la base unificada

Guía paso a paso para el equipo del Portal Deportivo · Club Atlético Camioneros de Córdoba · octubre de 2026

---

0. Qué se hace y por qué

Los dos portales pasan a usar **una sola base de datos SQL Server** y **un solo inicio de sesión** (el del Portal Administrativo). El Portal Administrativo es la fuente de verdad de la **deuda** de cada jugador; el Portal Deportivo le pregunta si el jugador puede entrar.

No hay que programar nada desde cero: les pasamos **dos parches** ya hechos y probados que modifican el código. Esta guía explica cómo aplicarlos, cómo configurar el proyecto y cómo comprobar que todo funciona.

Archivo	Para qué sirve
`ellos-definitivo.patch`	Parche del **BackEnd** (24 archivos: DAO, controladores, Program.cs, pruebas).
`ellos-definitivo-frontend.patch`	Parche del **FrontEnd** (15 archivos: alta de jugador, QR, pantalla de ingreso, sesión).
`aplicar-parche.ps1`	Script que aplica un parche cuidando los saltos de línea (CRLF/BOM) de sus archivos.
`ClubCamioneros_unificada_v2_crear_base.sql`	Script que crea la base unificada (vacía, con los catálogos).
`seed_cuentas_prueba.sql`	Cuentas de prueba (solo para desarrollo, nunca en producción).

Todo está en el repositorio **cacc-unificacion** (carpetas `parches/`, `base-de-datos/` y `pruebas/`).

1. Antes de empezar

1.1 Qué tiene que estar instalado

☐ Git para Windows.

☐ .NET SDK 9.

☐ Node.js 20.19 o superior y Google Chrome (para las pruebas del frontend).

☐ SQL Server (Express alcanza) y la herramienta `sqlcmd`.

☐ PowerShell (viene con Windows).

1.2 Partir de una copia limpia

Los parches están hechos sobre `main` en el commit `afa4e60`. Comprueben que están ahí y que no tienen cambios sin guardar:

```
cd C:\ruta\a\App-Club-Atletico-Camioneros
git checkout main
git pull
git log -1 --oneline
git status
```

Si el commit no es `afa4e60` (porque ya hicieron cambios), **avísennos antes de seguir**: puede que haya que regenerar el parche.

1.3 Crear una rama de trabajo

```
git checkout -b feat/base-unificada
```

1.4 Descargar los archivos de unificación

```
cd C:\ruta\donde\quieran
git clone <url del repo cacc-unificacion>
```

De acá en adelante, `UNIF` = la carpeta `cacc-unificacion` clonada.

2. Paso 1 · Crear la base de datos unificada

La base nueva **no se migra desde la vieja**: se crea vacía, con las tablas definitivas y los catálogos (roles, categorías, tipos de descuento). Los datos reales se cargarán después.

```
cd UNIF\base-de-datos
sqlcmd -S localhost -E -C -b -i ClubCamioneros_unificada_v2_crear_base.sql
```

- Crea la base `ClubCamionerosUnificadaV2`. Es seguro repetirlo: si ya tiene tablas, no hace nada.
- Si usan otro servidor o instancia: `-S localhost\SQLEXPRESS`.

Cuentas de prueba (solo en su máquina de desarrollo)

```
sqlcmd -S localhost -E -C -I -b -f 65001 -d ClubCamionerosUnificadaV2 -i UNIF\pruebas\seed_cuen
tas_prueba.sql
```

Crea estas cuentas. La contraseña de todas es `Camioneros.Test.2026!` y se entra **con el DNI**:

Usuario (DNI)	Rol	Para probar
20111222	SuperAdmin (ve los dos portales)	Altas, bajas, QR, categorías
30000003	Médica (solo Deportivo)	Lectura, sin permiso de altas
30000004	Director Técnico, categoría 3 (solo Deportivo)	Solo ve su categoría

Importante: el login ya no está en el Deportivo. Estas cuentas entran por el **Portal Administrativo** (paso 7).

3. Paso 2 · Aplicar el parche del BackEnd

3.1 Verificar que aplica (no modifica nada)

```
cd UNIF\parches
.\aplicar-parche.ps1 -Parche .\ellos-definitivo.patch -Carpeta C:\ruta\a\App-Club-Atletico-Cami
oneros\BackEnd -Verificar
```

Debe decir: *OK: el parche aplica (24 archivos). No se modificó nada.* Si falla, deténganse y avísennos con el mensaje de error.

3.2 Aplicarlo

```
.\aplicar-parche.ps1 -Parche .\ellos-definitivo.patch -Carpeta C:\ruta\a\App-Club-Atletico-Cami
oneros\BackEnd
```

Debe decir: *Parche aplicado: 24 archivos.* El script respeta el formato (CRLF/BOM) de cada archivo, así que en `git diff` solo van a ver los cambios reales.

Si PowerShell bloquea el script: `Set-ExecutionPolicy -Scope Process Bypass` y volver a ejecutarlo.

4. Paso 3 · Configurar el BackEnd

Creen (o editen) `BackEnd\CACC.API\appsettings.Development.json`. Este archivo **no debe subirse a git** (tiene la clave): su `.gitignore` hoy no lo excluye, así que agreguen la línea `appsettings.Development.json` a `BackEnd/.gitignore` (o al `.gitignore` de la raíz) y comprueben con `git check-ignore -v BackEnd/CACC.API/appsettings.Development.json`.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=ClubCamionerosUnificadaV2;Trusted_Connectio
n=True;TrustServerCertificate=True;"
  },
  "Jwt": {
    "Key": "<la MISMA clave que usa el Portal Administrativo>",
    "Issuer": "CACC-Unificado",
    "Audience": "CACC-Unificado"
  },
  "AdministrativoApi": {
    "BaseUrl": "http://localhost:5118",
    "TimeoutSegundos": 3
  }
}
```

Dato	Qué poner
`DefaultConnection`	La base creada en el paso 1. Ya no usen `Bbdd CACC.sql` ni la base `ClubCamioneros` vieja.
`Jwt:Key / Issuer / Audience`	**Tienen que ser idénticos** a los del Portal Administrativo (es quien emite el token, y ustedes lo validan). La clave se la pasa el equipo Administrativo por un canal privado; **nunca** se sube al repositorio. Mínimo 32 caracteres.
`AdministrativoApi:BaseUrl`	Dirección de la API del Administrativo. En desarrollo ya viene por defecto `http://localhost:5118`.
`AdministrativoApi:SiNoResponde`	Decisión del club: **si el Administrativo no responde, no pasa nadie**. Es el valor por defecto (`Denegar`); no hay que tocarlo.
`Cors:AllowedOrigins`	Opcional. Por defecto permite los puertos de desarrollo 4200 (Administrativo) y 4300 (Deportivo).

En producción, en lugar del archivo, se usan variables de entorno: `ConnectionStrings__DefaultConnection`, `Jwt__Key`, `Jwt__Issuer`, `Jwt__Audience`, `AdministrativoApi__BaseUrl`. La app **se niega a arrancar** en producción con la clave de ejemplo que hoy está en `appsettings.json`.

5. Paso 4 · Compilar y correr las pruebas del BackEnd

```
cd C:\ruta\a\App-Club-Atletico-Camioneros\BackEnd
dotnet build CACC.API\CACC.API.csproj
dotnet test CACC.Tests\CACC.Tests.csproj
```

Resultado esperado: compila sin errores y **56 pruebas correctas, 0 con error** (antes eran 18; se reescribieron las del QR porque ahora consulta al Administrativo).

6. Paso 5 · Aplicar el parche del FrontEnd y probarlo

```
cd UNIF\parches
.\aplicar-parche.ps1 -Parche .\ellos-definitivo-frontend.patch -Carpeta C:\ruta\a\App-Club-Atle
tico-Camioneros\FrontEnd -Verificar
.\aplicar-parche.ps1 -Parche .\ellos-definitivo-frontend.patch -Carpeta C:\ruta\a\App-Club-Atle
tico-Camioneros\FrontEnd
```

Debe decir que aplicó 15 archivos. Luego:

```
cd C:\ruta\a\App-Club-Atletico-Camioneros\FrontEnd
npm ci
npx ng build
npx ng test --watch=false --browsers=ChromeHeadless
```

Resultado esperado: el build termina bien (los avisos de "exceeded maximum budget" ya existían). Pruebas: **59 en total, 58 correctas**. La que falla es `admin-coordinador-legacy-routes` y ya fallaba **antes** del parche (nombres de clases); no la tienen que arreglar para esto.

Dirección del Portal Administrativo

Abran `FrontEnd\src\app\core\auth\portales.config.ts`. En desarrollo debe decir `http://localhost:4200` (así viene). **Al publicar**, cambiar por la dirección real del Administrativo.

7. Paso 6 · Levantar todo junto

Necesitan los dos portales corriendo, porque el Deportivo **ya no tiene login** y consulta la deuda al Administrativo. Hay cuatro procesos:

Qué	Dirección	Cómo se levanta
API Administrativo	http://localhost:5118	`dotnet run --project ApiGestion` (repo del Administrativo, rama `feat/base-unificada`). Con `Jwt:Key` igual a la del Deportivo y `ConnectionStrings:ConexionSQL` apuntando a la misma base.
Frontend Administrativo	http://localhost:4200	`cd frontend-cacc`, `npm ci`, `npx ng serve`
API Deportivo	http://localhost:5191	`dotnet run --project CACC.API` (en `BackEnd`)
Frontend Deportivo	http://localhost:4300	`npx ng serve --port 4300` (en `FrontEnd`)

Atajo: en `UNIF\scripts` hay un script que arma y levanta las dos APIs solas, con la base de prueba, para no configurar a mano:

```
.\armar.ps1 -RepoNuestro "C:\ruta\Sistema Administrativo - CACC" -RepoEllos "C:\ruta\App-Cl
ub-Atletico-Camioneros" -Recrear -Iniciar
.\probar.ps1      # 88 verificaciones automáticas contra las dos APIs
.\detener.ps1     # frena las APIs de prueba
```

(Trabaja sobre una copia: no modifica sus repositorios. Usa los puertos 5318 y 5319.)

8. Paso 7 · Probar a mano, de punta a punta

Con los cuatro procesos corriendo, hagan esta recorrida y tilden cada punto. **El pase de sesión entre los dos sitios es lo único que no está probado en un navegador:** si algo falla acá, avísennos con captura.

☐ Entrar a `http://localhost:4200` (Administrativo) con `20111222` y la contraseña de prueba. Aparece la elección de portales.

☐ Elegir **Portal Deportivo**: el navegador pasa a `localhost:4300`, entra sin pedir contraseña y la barra de direcciones **no muestra el token**.

☐ Abrir directamente `http://localhost:4300` en otra ventana sin sesión: muestra “El inicio de sesión es único” y un botón que lleva al Administrativo.

☐ Entrar con `30000003` (Médica) o `30000004` (DT): van **directo** al Portal Deportivo, sin pasar por la elección de portales.

☐ **Alta de jugador** (como SuperAdmin): el formulario tiene el campo **Género** obligatorio; sin género ni categoría no deja guardar.

☐ El jugador dado de alta aparece en el listado con estado de cuota (“AL DÍA”, “PENDIENTE” o “INHABILITADO”) que viene del Administrativo.

☐ **QR** (pantalla de escáner o DNI a mano): jugador al día → verde; con un mes de deuda → amarillo; con 2 o más cuotas vencidas → rojo y no registra asistencia.

☐ Escanear el mismo jugador dos veces el mismo día: dice “ya registrada” y no duplica.

☐ **Baja**: dar de baja un jugador desde el Deportivo. Desaparece de los listados, el QR lo rechaza (rojo, “dado de baja”) y sus pagos siguen en la base.

☐ **Categoría**: intentar borrar una categoría con jugadores. Muestra el error con el motivo y no la borra.

☐ **Apagar la API del Administrativo** y escanear un DNI: el QR responde que no se pudo consultar la deuda y **no deja pasar**; los listados siguen abriendo pero con “SIN VERIFICAR”.

☐ Cerrar sesión en el Administrativo: el token deja de servir también en el Deportivo (da 401).

9. Qué cambió en el código (para revisar)

Tema	Qué se hizo
Altas	`OUTPUT … INTO` y sin ids calculados a mano. Género (Masculino/Femenino) y categoría obligatorios en el formulario, en el servidor (400) y en la base. Al editar, el género nunca queda vacío.
Bajas	Listados, plantel y QR solo muestran jugadores activos. `DELETE /api/jugadores/{id}` ahora es **baja lógica** (`fecha_baja = hoy`); no borra pagos, descuentos, asistencias ni ficha médica.
Categorías	No se borra una categoría con jugadores o aranceles (HTTP 409). Ya no se deja `FK_id_categoria = NULL` .
Auditoría	Toda conexión de los DAO se abre con `OpenAuditadaAsync` , que registra quién escribe. **Regla para el futuro:** nunca abrir con `OpenAsync` a secas (una prueba falla si lo hacen).
Sesión	`Program.cs` valida en cada pedido que el usuario siga activo y que el token no esté invalidado. Cerrar sesión en un portal la cierra en ambos.
Login único	`POST api/auth/login` responde 410. La pantalla de ingreso recibe la sesión del Administrativo (`/login#sso=…`) y borra el token de la barra.
Roles	El rol 2 (Administrador del Administrativo) ya no entra al Deportivo.
QR y estado de cuota	La deuda ya **no se lee de** `VISTA_ESTADO_DEUDA` : se consulta a `GET /api/estado-deuda/dni/{dni}` del Administrativo con el token del usuario; los listados usan una sola llamada por lote. Si el Administrativo no responde: 503 y no pasa nadie.

10. Problemas frecuentes

Síntoma	Causa probable y solución
El parche "no aplica"	El repo no está en `afa4e60` o tiene cambios locales. Volver a una copia limpia de `main` o avisarnos.
Todo da 401 en el Deportivo	`Jwt:Key/Issuer/Audience` distintos a los del Administrativo, o la sesión se cerró (hay que volver a entrar por el Administrativo).
La app no arranca: "clave de ejemplo"	Está usando la clave de `appsettings.json` fuera de Development. Configurar `Jwt__Key` real.
El QR responde 503	El Administrativo no está corriendo o `AdministrativoApi:BaseUrl` es incorrecta. Probar `http://localhost:5118/api/estado-deuda/dni/20111222` con un token (debe dar 404 o 200, no "sin conexión").
El QR responde 401/403	El usuario no tiene permiso para consultar (el rol 2 no puede) o el token venció. Entrar de nuevo.
Error al dar de alta: género o categoría	La base rechaza altas sin género válido o sin categoría. Completar los dos campos.
CORS: el navegador bloquea las llamadas	El frontend corre en un puerto que no está en `Cors:AllowedOrigins`. Usar 4300 o agregarlo.
"Cannot find module" al compilar el frontend	Faltó `npm ci` en `FrontEnd`.

11. Cuando todo funciona: subir los cambios

```
cd C:\ruta\a\App-Club-Atletico-Camioneros
git status
git add -A BackEnd FrontEnd
git commit -m "feat: adaptacion a la base unificada y login unico (parche del Administrativo)"
git push -u origin feat/base-unificada
```

Después abran un Pull Request hacia `main`. **No suban** `appsettings.Development.json` ni ninguna clave.

12. Antes de publicar (producción)

☐ Cadena de conexión y `Jwt__*` por variables de entorno, con la clave real compartida con el Administrativo.

☐ `AdministrativoApi__BaseUrl` con la dirección pública del Administrativo.

☐ `Cors__AllowedOrigins__0` con el dominio real del frontend del Deportivo.

☐ `portales.config.ts` con la dirección real del Administrativo (y, del otro lado, nosotros completamos la del Deportivo).

☐ Las cuentas de prueba **no existen** en producción.

¿Algo no coincide con lo que ven? Mándennos el mensaje de error exacto y el número de paso. Detalle técnico completo: `docs/parche-deportivo.md` y `docs/hallazgos-otro-equipo.md`.